

Week 9: Lab

Solution

2025-10-21

Load packages

```
rm(list=ls())

pkgs <- c("tidyverse", "dplyr", "readr", "magrittr", "ggplot2", "readxl",
          "cowplot", "GGally", "knitr", "kableExtra", "data.table",
          "skimr", "flextable", "Metrics", "devtools", "qqman", "rrBLUP")
for (i in seq_along(pkgs)){
  pkg <- pkgs[[i]]
  tryCatch(if(pkg%in%rownames(installed.packages())){
    cat("Available:", pkg, "\n")
    library(pkg, character.only = TRUE)
    cat("Package loaded:", pkg, "\n")
  }
  else if(!(pkg%in%rownames(installed.packages()))){
    install.packages("BiocManager")
    BiocManager::install(pkg)
    library(pkg, character.only = TRUE)
    cat("Package loaded:", pkg, "\n")
  }
  else{
    cat("Not available:", pkg, ">> now installing...", "\n")
    install.packages(pkg, repos = "https://cloud.r-project.org")
    cat("Now available:", pkg, "\n")
    library(pkg, character.only = TRUE)
    cat("Package loaded:", pkg, "\n")
  }
}
```

```
)  
}
```

Paths

```
plk_path #PLINK path  
f_path #Files path
```

Convert to PLINK format

```
#####  
## To use PLINK, you need to use its command style which includes:  
##  
## --file [reads .ped and .map files]  
## --make-bed [converts the two files (.map and .ped) to PLINK format (.bam,  
##           .bim, and .fam) files]  
## --out [provides output files]  
## --bfile [reads the PLINK format files to perform analysis]  
## --geno [set the threshold for maximum missing per variants]  
## --mind [sets the threshold for maximum missing per sample]  
## --freq [produces the allele frequency per variants]  
## --maf [sets the threshold for minor allele frequency]  
## --make-rel square [performs the genomic relationship matrix in a square  
##                  matrix]  
#####  
  
#####  
## The use of sprintf() is to make the code smooth  
## %s means we are calling a string objects that we set in lines 52-53 above  
## So, each %s relates to the objects in order after the comma  
#####  
  
# Convert .ped and .map files to plink format (.fam, .bed, and .bam)  
  
cmd <- sprintf("%s --file %s/DLY_60K_gens --make-bed --out %s/out_plink_DLY",  
               plk_path, f_path, f_path)  
system(cmd)
```

```
[1] 0
```

Quality control

```
#####  
## Check the allele frequencies before MAF  
#####  
  
cmd_freq <- sprintf("%s --bfile %s/out_plink_DLY --freq --make-bed --out %s/DLY_freq",  
                    plk_path, f_path, f_path)  
system(cmd_freq)
```

```
[1] 0
```

```
## Remove unnecessary files. You don't need these PLINK format files in this QC  
  
file.remove(sprintf("%s/DLY_freq.bim", f_path))
```

```
[1] TRUE
```

```
file.remove(sprintf("%s/DLY_freq.bed", f_path))
```

```
[1] TRUE
```

```
file.remove(sprintf("%s/DLY_freq.fam", f_path))
```

```
[1] TRUE
```

```
#####  
## Obtain minor allele frequencies  
#####  
  
cmd_maf <-  
  sprintf("%s --bfile %s/out_plink_DLY --geno 0.05 --maf 0.05 --make-bed --out %s/DLY_maf",  
          plk_path, f_path, f_path)  
system(cmd_maf)
```

```
[1] 0
```

```
#####
## Check the allele frequencies after MAF
#####

cmd_freq2 <- sprintf("%s --bfile %s/DLY_maf --freq --make-bed --out %s/DLY_frq2",
                    plk_path, f_path, f_path)
system(cmd_freq2)
```

```
[1] 0
```

```
## Remove unnecessary files. You don't need these PLINK format files in this QC
file.remove(sprintf("%s/DLY_frq2.bim", f_path))
```

```
[1] TRUE
```

```
file.remove(sprintf("%s/DLY_frq2.bed", f_path))
```

```
[1] TRUE
```

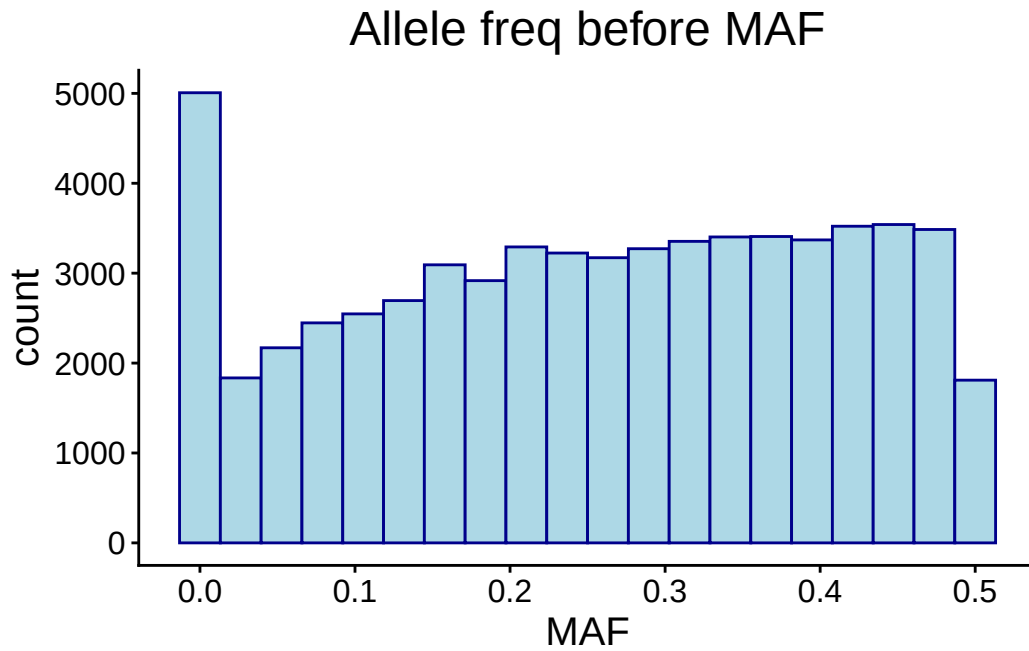
```
file.remove(sprintf("%s/DLY_frq2.fam", f_path))
```

```
[1] TRUE
```

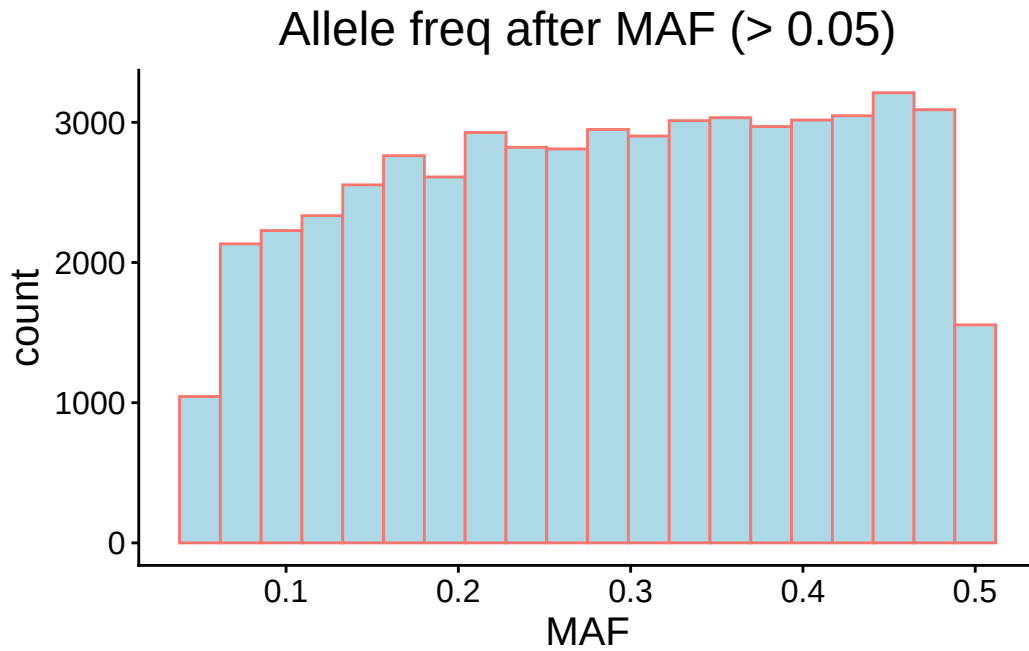
```
#####
## Plot the allele frequencies before and after MAF
#####

freq1 <- fread(sprintf("%s/DLY_frq.frq", f_path)) #before MAF
freq2 <- fread(sprintf("%s/DLY_frq2.frq", f_path)) #after MAF

par(mfrow=c(1,2))
ggplot(freq1, aes(x=MAF))+
  geom_histogram(bins = 20, colour = "darkblue", fill="lightblue")+
  ggtitle("Allele freq before MAF")+
  theme_classic()+
  theme(plot.title = element_text(hjust=0.5), text=element_text(size=15))
```



```
ggplot(freq2, aes(x=MAF))+  
  geom_histogram(bins = 20, colour = "#F8766D", fill="lightblue")+  
  ggtitle("Allele freq after MAF (> 0.05)")+  
  theme_classic()+  
  theme(plot.title = element_text(hjust=0.5), text=element_text(size=15))
```



```
## Summary statistics and dimension
summary(freq1$MAF)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0000  0.1322  0.2631  0.2546  0.3842  0.5000
```

```
summary(freq2$MAF)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0500  0.1820  0.2934  0.2885  0.3992  0.5000
```

```
length(freq1$MAF)
```

```
[1] 61565
```

```
length(freq2$MAF)
```

```
[1] 53009
```

PLINK GRM

```
## Compute genomic relationship matrix and set missing individual to 0.05
## and minor allele frequency to 0.05
```

```
cmd_grm <-
  sprintf("%s --bfile %s/DLY_maf --mind 0.05 --maf 0.05 --make-rel square --out %s/DLY_grm",
          plk_path, f_path, f_path)
system(cmd_grm)
```

```
[1] 0
```

```
#Read the genomic relationship matrix
```

```
DLY_grm <- fread(sprintf("%s/DLY_grm.rel", f_path))
dim(DLY_grm)
```

```
[1] 607 607
```

```
DLY_grm[1:5, 1:5]
```

	V1	V2	V3	V4	V5
	<num>	<num>	<num>	<num>	<num>
1:	0.9152000	0.1378340	-0.01665390	0.07111330	0.20207700
2:	0.1378340	0.9338540	-0.01004960	-0.03746650	0.12897400
3:	-0.0166539	-0.0100496	0.87359200	0.00809403	0.00190767
4:	0.0711133	-0.0374665	0.00809403	0.88385100	-0.00140690
5:	0.2020770	0.1289740	0.00190767	-0.00140690	0.92217100

```
#Read the genomic relationship matrix IDs
```

```
ddgrm_id <- fread(sprintf("%s/DLY_grm.rel.id", f_path))
head(ddgrm_id)
```

	V1	V2
	<int>	<char>
1:	3	DLY0091
2:	5	DLY0092

```

3:      6 DLY0093
4:      7 DLY0094
5:     11 DLY0095
6:     15 DLY0096

```

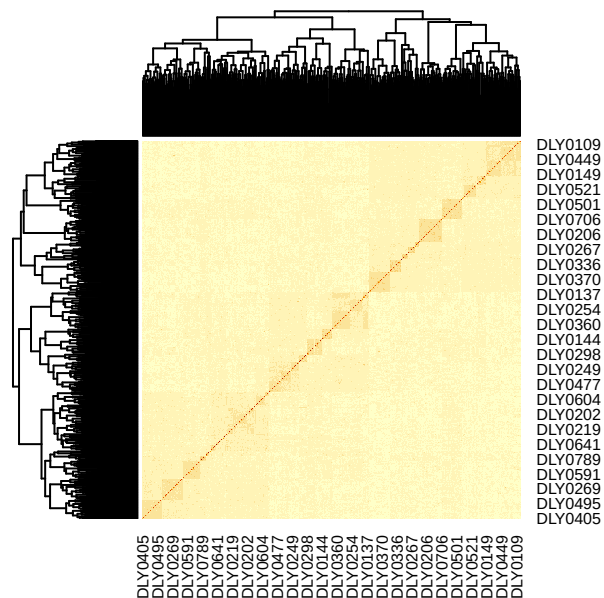
Convert grm to matrix and add row and column names

```

gmxt <- as.matrix(DLY_grm) #convert GRM to a matrix
colnames(gmxt) <- ddgrm_id$V2 #name the columns using the GRM IDs
rownames(gmxt) <- ddgrm_id$V2 #name the columns using the GRM IDs

heatmap(gmxt) #plot the heatmap for the GRM

```



Manual GRM calculation

Read allele count

```

## Recode to 0/1/2

## Read the allele file

DLY_rec <- fread(sprintf("%s/DLY_recode.raw", f_path))|>
  select(-c("FID", "PAT", "MAT", "SEX", "PHENOTYPE")) |> #remove unnecessary columns
  column_to_rownames(var="IID") # convert the IID column to rownames

## Remove missing values

nomis <- (na.omit(t(DLY_rec)))
nomis2 <- (na.omit(t(nomis)))

## Load this necessary file
load(sprintf("%s/DLY_recode2.RData", f_path))

```

GRM function

```

#####
## GRM function
## It converts input to matrix, removes minor allele frequencies (MAF) less
## than 0.05, and computes the GRM
## It returns allele frequencies before (freq0) and after (freq) MAF,
## MAF, genotype corrected for MAF (gen01), and
## genomic relationship matrix (GRM)
## Because the return is a list, you'll add $ to the object to obtain any of
## GRM function object you need
#####

GRM <- function(x){x1 <- as.matrix(x); p0 <- colMeans(x1)/2;
maf <- pmin(p0,1-p0) >0.05; gen <- x1[,maf]; p1 <- colMeans(gen)/2;
z0 <- sweep(gen, 2, 2*p1)/sqrt(sum(2*p1*(1-p1))); G0 <- tcrossprod(z0);
return(list(GRM=G0, freq=p1, zmat=z0, freq0=p0, gen01=gen, maf=maf))}

#####
## Calculate genomic relationship matrix (GRM)
#####

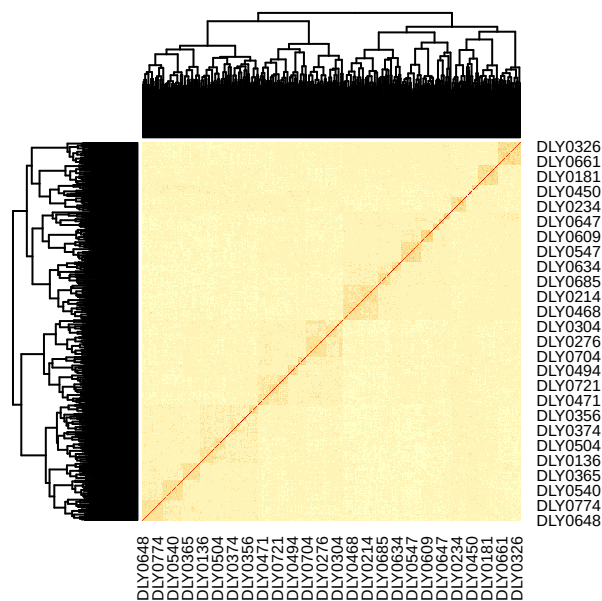
grm_a <- GRM(nomis3)

```

```
#This gives the summary statistics of the main diagonal of GRM
summary(diag(grm_a$GRM))
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.7808  0.8543  0.8892  0.8900  0.9230  1.2708
```

```
# This gives the heatmap for the GRM.
# Deeper colors indicate higher relationship between two variants and lighter
# colors indicate lower relationship
heatmap(grm_a$GRM)
```



Read the phenotype

```
pheno <- fread(sprintf("%s/DLY_60K_phens.txt", f_path))

## If you use manual calculation, change gmxt to grm_a$GRM below

## This includes id, sex, batch, and a phenotype (HDL-C/LDL-C)
pheno2 <- pheno[pheno$id%in%rownames(gmxt)]|>select(1:3,8)
```

```
## This includes id, sex, batch, and a phenotype (AI)
pheno3 <- pheno[pheno$id%in%rownames(gmxt)]|>select(1:3,9)
```

Polygenic score

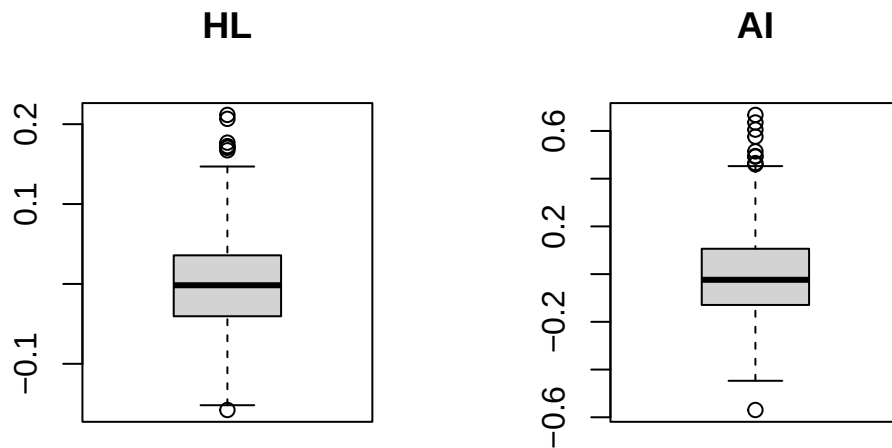
```
## Computes the X matrix for fixed effects
Xm <- model.matrix(~as.factor(pheno2$sex)+pheno2$batch)

## Perform the linear mixed model to obtain the polygenic scores

prs_HL <- mixed.solve(pheno2$`HDL-C/LDL-C`, X=Xm, K= grm_a$GRM)
prs_AI <- mixed.solve(pheno3$AI, X=Xm, K= grm_a$GRM)

## Plot the polygenic scores

par(mfrow=c(1,2))
boxplot(prs_HL$u, main="HL")
boxplot(prs_AI$u, main="AI")
```



Read the map file

```
## Read the map file
map_f <- fread(sprintf("%s/DLY_60K_gens.map", f_path))
head(map_f) # view the first six rows of the map file
```

	V1	V2	V3	V4
	<int>	<char>	<int>	<int>
1:	0	ALGA0000474	0	0
2:	0	ALGA0001018	0	0
3:	0	ALGA0001019	0	0
4:	0	ALGA0001427	0	0
5:	0	ALGA0001428	0	0
6:	0	ALGA0001436	0	0

```
## Remove the unnecessary character in the genotype object
## (grm_a$gen01) column names
```

```
head(colnames(grm_a$gen01))
```

```
[1] "ALGA0000474_1" "ALGA0001018_1" "ALGA0001019_1" "ALGA0002043_1"
[5] "ALGA0002567_1" "ALGA0004243_1"
```

```
# you'll notice extra characters in the colnames which we are not in the map object
# variant names. So we want to remove them to ensure the variant names in our genotype
# matches those in the map object
```

```
hnomis2 <- as.character(sub("_.*", "\\1", colnames(grm_a$gen01)))
```

```
## Subset the map object to match the variants in the genotype object
mapp2 <- map_f[map_f$V2%in%hnomis2,]
```

Obtain the genotype file

- This is the input to perform gwas using rrBLUP

```
geno_io <- data.frame(marker = mapp2[, 2], chrom = mapp2[, 1],
                     pos = mapp2[,4], t(grm_a$gen01 - 1), check.names = FALSE)
```

Perform GWAS

```
#####  
## GWAS using rrBLUP GWAS function  
#####  
  
## GWAS function output -log10 pvalues so we reconvert to original pvalues using mutate()  
## We don't need chromosome 0 and 23, so we filter it out (remove it) using filter()  
## We rename the output file to match manhattan plot input structure using names()  
  
## Here we have three models to examine if the simple model (only fixed effect)  
## is just fine than models including GRM and GRM+population structure (PC)  
  
## Only fixed effects  
  
gwas1 <- GWAS(pheno2, geno_io, fixed = c("sex", "batch"),  
              min.MAF = 0.05, plot = FALSE)
```

```
[1] "GWAS for trait: HDL-C/LDL-C"  
[1] "Variance components estimated. Testing markers."
```

```
gwas1a <- gwas1|>  
  mutate(HDL.C.LDL.C=10^(-HDL.C.LDL.C))|> #Backtransform -log10 pvalues  
  filter(V1!="0" & V1!="23") #remove CHR 0 and 23  
names(gwas1a) <- c("SNP", "CHR", "BP", "P") #rename the output file  
  
## Fixed effects and relationship matrix  
  
gwas2 <- GWAS(pheno2, geno_io, fixed = c("sex", "batch"), K=grm_a$GRM,  
              min.MAF = 0.05, plot = FALSE)
```

```
[1] "GWAS for trait: HDL-C/LDL-C"  
[1] "Variance components estimated. Testing markers."
```

```
gwas2a <- gwas2|>  
  mutate(HDL.C.LDL.C=10^(-HDL.C.LDL.C))|> #Backtransform -log10 pvalues  
  filter(V1!="0" & V1!="23") #remove CHR 0 and 23  
names(gwas2a) <- c("SNP", "CHR", "BP", "P") #rename the output file  
  
## Fixed effects, relationship matrix, and population structure
```

```
gwas3 <- GWAS(pheno2, geno_io, fixed = c("sex", "batch"), K=grm_a$GRM, n.PC=4,
             min.MAF = 0.05, plot = FALSE)
```

```
[1] "GWAS for trait: HDL-C/LDL-C"
[1] "Variance components estimated. Testing markers."
```

```
gwas3a <- gwas3|>
  mutate(HDL.C.LDL.C=10^(-HDL.C.LDL.C))|> #Backtransform -log10 pvalues
  filter(V1!="0" & V1!="23") #remove CHR 0 and 23
names(gwas3a) <- c("SNP", "CHR", "BP", "P") #rename the output file
```

```
## Sort the -log10 pvalues (greater than 0) in descending order
```

```
head(sort(gwas1$HDL.C.LDL.C[gwas1$HDL.C.LDL.C!="0"], decreasing = TRUE))
```

```
[1] 5.577334 4.786352 4.561886 4.013640 3.885589 3.714653
```

```
head(sort(gwas2$HDL.C.LDL.C[gwas2$HDL.C.LDL.C!="0"], decreasing = TRUE))
```

```
[1] 5.577334 4.786352 4.561886 4.013640 3.885589 3.714653
```

```
head(sort(gwas3$HDL.C.LDL.C[gwas3$HDL.C.LDL.C!="0"], decreasing = TRUE))
```

```
[1] 5.420234 4.601553 4.308563 3.989785 3.965364 3.776787
```

```
## Plot the manhattan plot for the three GWAS models
## Note the blue horizontal line is the suggestive significant threshold
## (-log10(1e-5)) and not the genome-wide threshold (which would indicate
## stronger association)
```

```
par(mfrow=c(2,3))
manhattan(gwas1a,col = c("dark blue","cornflowerblue"),
          main="HDL.C.LDL.C w/Fe")
qq(gwas1a$P,col = "dark blue", main="HDL.C.LDL.C w/Fe")
manhattan(gwas3a,col = c("dark blue","cornflowerblue"),
          main="HDL.C.LDL.C w/Fe+K")
qq(gwas2a$P,col = "dark blue", main="HDL.C.LDL.C w/Fe+K")
manhattan(gwas3a,col = c("dark blue","cornflowerblue"),
          main="HDL.C.LDL.C \n w/Fe+K+PC")
qq(gwas3a$P,col = "dark blue", main="HDL.C.LDL.C \n w/Fe+K+PC")
```

